

Attributing Steering Effects to Specific Dictionary Slots

A control protocol for a frozen language model, and what survived auditing it

Amr Elsaed Independent Researcher amralsaed225@gmail.com

September 2026

Code and data availability. All experiment sources, generated notebooks, and the complete run logs for every experiment reported here (29 runs) are available at: <https://github.com/aammr3/sparse-dictionary-attribution>. The research log documenting all 79 recorded events, including the four withdrawn claims of §5, is included in that repository, in the original Arabic and in English translation.

Abstract

When writing into a sparse dictionary's slots changes a frozen language model's output, what licenses the claim that the *concept* those slots encode caused the change? We give a control protocol for that question, run it, and report what it licenses and what it does not.

The question is live because the field currently has no agreed way to determine whether a given dictionary's handles are meaningful: recent work reports that randomized dictionary variants match trained ones on standard evaluations [1], an audit of a widely used benchmark finds several of its metrics unfit for use [2], and automated interpretability metrics have been shown not to distinguish trained from randomly initialized models [3]. The diagnosis is better developed than the remedy.

Our main result. On a frozen Pythia-410M with a TopK concept registry (4096 slots, $k = 16$) over four text domains, a targeted intervention rises monotonically to 41.7% on a concept-defined metric while **three independent controls return exactly zero at every strength**: a magnitude-matched random direction, quality-matched features belonging to other concepts, and a **shuffled-decoder** control that biases the same slots through the same learned columns under a permuted assignment. The last of these is the informative one, and we did not find it in use in this literature. Subject to a dose mismatch we report rather than suppress (18–29% against a 5% tolerance), the pattern is most consistent with the effect arising from the specific slot-to-column correspondence rather than from slot quality or from the columns merely being learned.

Why the controls are the contribution. Getting to a result we could defend required removing several we could not. We document **six evaluation pitfalls**, each quantified from our own runs, that individually produced a false negative or a false positive in our results: uncalibrated intervention magnitude, greedy decoding artifacts, metric mismatch, measurement at the wrong token span, a control pinned at the metric floor, and a metric that rewards fluency collapse. Four produced effects we recorded as absent; two produced numbers we recorded as evidence and later withdrew.

We then subject our own settled results to an adversarial audit, which **overturns or restricts four recorded claims**, including the one our internal record described as the cleanest result in the project. We report what survives, and note that the two false positives could not have been found by adding instrumentation — the instruments were reporting correctly; the inference drawn from them was wrong.

We organize the control landscape by what each class rules out, and identify two constructions we did not find used in this literature: controls matched on a *measured feature-quality score*, and the shuffled-decoder control above (permutation controls are long established in other settings). We further report that the feature-defined metric ranks a random direction *above* the intended intervention in one domain — an independent confirmation of the sixth pitfall, on new data.

This is not a novelty claim. Where prior work anticipates our findings we say so. Its contribution is that the failure modes are measured rather than hypothesized, and reported by the people who fell into them.

Keywords: sparse autoencoders · activation steering · evaluation methodology · causal specificity · controls · negative results

Contributions

1. **An attribution result** (§6.6): a steering effect that survives three independent controls at zero, including a shuffled-decoder control we did not find in use in this literature — reported with the dose mismatch that limits it.
2. **Six quantified evaluation pitfalls** from the same pipeline (§4), each with the reasoning that produced the wrong conclusion, the measurement that revealed it, and a transferable rule.
3. **An adversarial audit of our own settled results** (§5), reported in full, which overturned or restricted four claims and whose selectivity is itself diagnostic.
4. **A control taxonomy** organized by what each class rules out (§6.2), with two constructions we did not find used in this literature (§6.3), four of five run empirically (§6.6).
5. **A diagnostic for null results** (§4.4): when a null includes a baseline you expected to work, the joint failure localizes the problem to the measurement position rather than the instrument. Applying it converted a complete null into the strongest positive result in our pipeline.
6. **Results that survive the audit** (§3), stated with the restrictions it imposed — including a frozen model's dictionary state at entity-read time discriminating real from invented entities better than the model's own next-token uncertainty at the same positions, in one of three non-confounded domains, at $n = 30$ per class and a single training seed.

1. Introduction

Sparse dictionaries trained on the activations of a frozen language model are now a standard tool for obtaining interpretable, and putatively causal, handles on model internals. The premise is that writing into a dictionary's slots constitutes an intervention on a *concept*, and that the resulting behavioural change can be attributed to that concept.

This paper is about the second half of that premise. Demonstrating that an intervention changes behaviour is routine. Demonstrating that the change is attributable to the intervened feature — rather than to the act of intervening, to the magnitude injected, or to the feature merely being a learned one — requires controls that rule out each of those alternatives separately. We set out what those controls are, run four of them, and report the attribution they license.

That premise is currently under pressure. Recent work reports that randomized dictionary variants match fully-trained ones on interpretability scores, sparse probing, and causal editing, and concludes that dictionaries in their present form "do not reliably decompose models' internal mechanisms" [1]. A separate audit of a widely used benchmark suite [7] finds that several of its metrics "should not be used to evaluate SAEs," and proposes no replacement [2]. A third line of work finds that automated interpretability metrics fail to distinguish trained from randomly initialized transformers at all [3].

The common thread is not that dictionaries do not work. It is that **the field does not currently have an agreed way to tell whether a given dictionary's handles are meaningful**, and the metrics in use are too weak to separate learning from the exploitation of random structure at scale.

What this paper contributes

We do not resolve that question. We report, from a complete pipeline built to answer a different question, a set of concrete failure modes that lie between a correct implementation and a correct conclusion — and a self-audit that removed four of our own results.

Specifically:

1. **Six measured evaluation pitfalls** (§4), each quantified from our own runs. Four produced false negatives — effects we recorded as absent that were present. Two produced false positives — numbers we recorded as evidence that were artifacts. Each is presented with the reasoning that led us to the wrong conclusion, the measurement that revealed it, and the transferable rule.
2. **An adversarial audit of our own settled results** (§5), which overturned or restricted four recorded claims, including the one we had described in our internal record as the cleanest result in the project. We report the audit's method, its findings, and — because it is diagnostic — the observation that the two false positives could not have been caught by adding instrumentation, only by attacking conclusions we already believed.
3. **A control taxonomy and a proposed protocol** (§6), organized around what each control class actually rules out. We identify two constructions for which we did not find in use in this literature — controls matched on a *measured feature-quality score*, and a *shuffled-decoder* control that preserves the learned columns but permutes their assignment — run four of the five classes (§6.6), and show from our own data why the quality-matched control alone carries no information.
4. **The results that survive** (§3), stated with the restrictions the audit imposed. The strongest of these is that a frozen model's sparse-dictionary state, read at the moment it processes an entity name, discriminates real from invented entities better than the model's own next-token uncertainty at the same positions — in one of three non-confounded domains.

What this paper is not

It is not a novelty claim. The mechanisms we build on are established, and where prior work anticipates our findings we say so explicitly (§8) — including one paper that reports entity-recognition directions of the kind we recover in §3.5 [4], and another that separates causal read- from write-inertness along the same lines as our own mechanistic reasoning [8].

It is a case study, on a single 410M-parameter model, four text domains, and one dictionary architecture. Its value, if it has any, is that the failure modes are measured rather than hypothesized, and that they are reported by the people who fell into them.

Why report failures at this level of detail

Each pitfall in §4 cost us at least one full experimental run, and in one case five. Two of them produced results we recorded internally as successes and later withdrew. None of them are exotic: uncalibrated intervention magnitude, greedy decoding, a metric that does not measure what the mechanism moves, measurement at the wrong token positions, a control that cannot vary, and a metric that rewards degenerate output.

We expect these to be common precisely because they are individually easy to miss and collectively invisible in published work — a paper that fell into any of them would report either a clean null or a clean effect, and in neither case would the underlying failure be visible to a reader.

2. Setup

2.1 Model and dictionary

Base model. Pythia-410M [14] (EleutherAI), frozen throughout. Every parameter has `requires_grad = False`, the model is held in evaluation mode, and the optimizer is constructed over dictionary parameters only. The base model's next-token cross-entropy was measured on held-out data at intervals during training and remained stable, confirming that no parameter drift occurred.

Dictionary. A TopK sparse dictionary ("registry") over residual-stream activations at layer 12 of 24:

```
pre  = ReLU( W_enc (h - b_dec) )
acts = TopK( pre, k )
recon = W_dec acts + b_dec
```

with `n_slots = 4096`, `k = 16`, and an auxiliary top-`k_aux = 32` reconstruction term (weight 0.03) to revive slots that have stopped firing. Decoder columns are unit-normalized at initialization and renormalized periodically during training. The training objective is reconstruction only — no domain labels enter it.

The expansion factor is 4 (4096 slots over 1024 dimensions). This is substantially lower than is typical in recent dictionary work, and we treat it as a scope limitation rather than a design choice (§7).

Domains. Four text corpora: medicine (PubMedQA), law (BillSum), code (CodeParrot), literature (Project Gutenberg). Domains define the concept partition against which slot specialization is measured; they are not used as training signal.

2.2 Measuring specialization

For each slot we count firings per domain over held-out text and define

```
purity(slot) = max_d count[d] / Σ_d count[d]
```

so a slot firing only within one domain has purity 1.0 and a slot firing equally across four has purity 0.25.

The comparison of interest is against the model's own hidden dimensions. To be meaningful, this must be made **at matched sparsity**: raw-dimension purity is computed over the top-16 dimensions per token, the same budget the dictionary operates under. Comparing a sparse representation against a dense one inflates the apparent gap and we do not report such comparisons.

2.3 Intervention

Steering biases slot scores *before* selection, so that the bias can change which slots are selected, and injects the difference between the two decodes as a residual-stream update at the same layer:

```
delta = decode( TopK(pre + bias) ) - decode( TopK(pre) )
h'    = h + gain · delta
```

At `bias = 0` this is a bit-exact no-op, verified by the fact that purity and training metrics are identical to the runs without a steering hook installed.

Magnitude. Every intervention row reports

```
delta_ratio = || gain · delta || / || h ||
```

the norm of the injected update relative to the residual stream it is added to, averaged over token positions. §4.1 explains why we regard reporting this as mandatory rather than optional.

2.4 Generation and fluency gating

Generation uses sampling (`temperature = 0.8` , `top_p = 0.9`). Greedy decoding is not used anywhere in this work; §4.2 explains why. The nucleus filter includes the token that crosses the probability threshold rather than excluding it.

Every generated continuation is scored for repetition rate and mean token-level entropy, and rows exceeding a repetition threshold or falling below an entropy threshold are flagged as degenerate. §4.6 documents a case where a metric was maximized by exactly such degenerate output, and §5.2 records the consequence.

2.5 Metrics

- **Concept-defined rate** — frequency of terms selected as distinctive to a domain by relative frequency against the other three corpora.
- **Feature-defined rate** — frequency of the tokens the intervened slots themselves most strongly fire on. Introduced as a mismatch diagnostic (§4.3), and restricted to that role after §4.6.
- **AUROC**, polarity-corrected as `max(AUC, 1 - AUC)` , for the detection experiments. Polarity correction is necessary because several registry signals are inverted-but-informative; it is applied identically to registry signals and to baselines, and the permutation test in §3.5 recomputes the maximum over signals inside every shuffle so that the correction cannot inflate the reported significance.

2.6 Detection experiment

For the hallucination experiments the steering bias is identically zero throughout and the hook is read-only; this is asserted at six points in the execution path. Signals are recorded over three spans — the entity tokens specifically, the full prompt, and the generated tokens — with the entity span located by re-encoding the template prefix, prefix-plus-entity, and the full prompt, and verifying that the reconstruction agrees.

LM-head baselines (next-token entropy and negative max log-probability) are computed from the prefill at every context position, from the full softmax before temperature and nucleus filtering, and sliced into the same spans under two alignments: at the same positions as the registry signal, and shifted by one so as to characterize the same tokens. The verdict is required to clear **the stronger of the two**, so that the choice of alignment cannot favour the dictionary.

2.7 Reproducibility of the runs reported here

Training in this configuration is bit-reproducible on the hardware used (NVIDIA T4). Across ten independent sessions the purity gap was identical to sixteen significant figures and per-slot firing counts were identical as integers. All runs used a single fixed training seed; consequently seed-to-seed variance was never measured, and no claim in this paper rests on stability across seeds (§3.2, §5.2 R4).

3. Results

This section reports the findings that survive the adversarial audit described in §5. Four claims that appeared in earlier internal versions of this work were withdrawn or restricted by that audit; they are listed in §5.2 rather than here, and the restrictions are carried into the statements below.

Every number is traceable to a run log; references are given inline.

3.1 Slots specialize without supervision

Trained with a reconstruction objective alone — no domain labels, no auxiliary classification term — registry slots separate by domain far more cleanly than the model's own hidden dimensions do at the same sparsity.

	registry slots	raw hidden dims
mean purity	0.805	0.583
gap	+0.222	—

The comparison is made at matched sparsity: the raw-dimension purity is computed over the top-16 dimensions per token, the same budget the registry is given. Without this matching the comparison is not meaningful, since a dense representation is being compared against a sparse one.

The effect is monotone in dictionary capacity, over three successive configurations at fixed sparsity `k = 16` :

slots	1024	2048	4096
mean purity	0.766	0.786	0.805

A hypothesis we rejected. Our prior expectation was the opposite of what we found. We expected that *reducing* the number of simultaneously active slots would force cleaner specialization, and tested `k = 8` at fixed capacity. Every measured quantity degraded: mean purity fell to 0.708, the purity gap fell from 0.187 to 0.089, and language-model loss worsened. Increasing capacity, not increasing sparsity pressure, is what produced specialization in our setting.

3.2 The mechanism transfers to a frozen pretrained model, and improves with scale

We repeated the procedure on frozen Pythia models, training only the registry on mid-network activations while every model parameter was held fixed (`requires_grad = False`, verified; cross-entropy of the base model measured stable throughout training).

	Pythia-160M	Pythia-410M
purity gap over raw dims	+0.288	+0.317
dead slots at end of training	5.1%	0.5%
live slots	2377 / 4096	3527 / 4096

Both the separation and the utilization of the dictionary improve with model scale. Two data points do not establish a trend, and we do not claim one; we report the direction and note that it is consistent with the expectation that richer representations admit cleaner sparse decomposition.

Reproducibility. Training in this regime is bit-reproducible on the hardware used. Across **ten independent sessions spanning seven distinct codebases**, the reported purity gap was identical to sixteen significant figures (`0.31747859716415405`), as were per-slot firing counts as integers. We note this explicitly because an earlier phase of this

work — in which the dictionary was trained *jointly* with a from-scratch transformer — was strongly non-reproducible at fixed seed, and we had incorrectly carried that expectation forward. See §5.2, R4.

A caveat follows directly from the same fact: because the seed was held fixed at a single value throughout, **seed-to-seed variance in this regime has never been measured**. No claim in this paper depends on stability across seeds.

3.3 Writing into slots changes generation, monotonically in the injected magnitude

Biasing a domain's purest slots before top-k selection, and injecting the resulting difference of decodes as a residual-stream update, produces a monotone dose–response relationship between the relative magnitude of the update and the rate of targeted vocabulary in the continuation:

delta_ratio	0%	2.8%	6.1%	7.7%	9.0%	11.4%
targeted-vocabulary rate	0.0%	1.3%	16.0%	22.8%	38.2%	41.7%

Monotonicity across six points is the relevant evidence here, not any single value; individual cell values carry substantial variance (§5.2, R1).

Fluency was checked on every row via repetition rate and token-level entropy, under sampling-based decoding. The operating region in which the intervention produces measurable movement without degrading fluency is approximately `delta_ratio ∈ [6%, 8%]`; below roughly 1% the intervention is not detectable (§4.1), and above roughly 10% generation degrades.

3.4 Suppression is structurally saturated

The same mechanism applied in the negative direction — biasing a domain's slots *downward* so they fall out of the top-k selection — does not produce a comparable intervention. Measured at identical settings:

direction	maximum achieved delta_ratio
promotion	12.29%
suppression	≤ 0.36%

a factor of **34**, and the suppression figure does not increase with strength. The mechanism is straightforward: removing k slots from a top-k selection promotes the next k in rank order, and their reconstruction is nearly equivalent, so the difference between the two decodes remains small no matter how negative the bias becomes.

This is a limitation of the intervention mechanism, not of the dictionary. Any application requiring suppression rather than promotion needs a different mechanism — for example forcing activations to zero after selection rather than biasing scores before it.

3.5 The registry's internal signature at entity tokens exceeds the model's own uncertainty signal

We tested whether the registry's internal state predicts ungroundedness *before generation*, using a read-only hook (steering bias identically zero throughout, asserted at six points in the code) on a frozen Pythia-410M.

The contrast is between prompts naming real entities and prompts naming invented ones, where the invented case is ungrounded by construction. Signals are computed over three spans — the entity tokens, the full prompt, and the generated tokens — and compared against the model's own LM-head baselines (next-token entropy and negative max

log-probability) computed at the **same positions**, in two alignments, with the verdict required to clear the stronger of them.

Automated verdict. At the entity span, the best registry signal exceeds the strongest of four LM-head baselines by **+0.139** discrimination (*logs27.txt:1121*). At the generated-token span the margin is +0.047, close to the noise floor — consistent with §4.4, where measuring only at generated tokens returned a complete null.

Independent entity-level analysis. Because the entity-span signals are read from the deterministic prefill, they are bit-identical across generation repeats — verified for all 240 entities. The effective independent unit is therefore the entity, not the run, and any interval computed over runs would be too narrow by a factor of $\sqrt{3}$. Re-analysing at the entity level (30 real vs 30 invented per domain), aggregating each signal over the whole entity span so that no threshold on span length is selected post hoc:

domain	best registry signal	disc.	strongest baseline	disc.	gap	95% CI	perm. <i>p</i>	signals beating baseline
literature	prompt_domain_share	0.837	neg_max_logprob_state	0.636	+0.201	[+0.020, +0.344]	0.0015	5 / 7
law	max_act	0.750	lm_entropy_token	0.623	+0.127	[-0.070, +0.267]	0.0285	2 / 7
code	max_act	0.653	neg_max_logprob_state	0.713	-0.060	[-0.149, +0.128]	0.93	0 / 7
~~medicine~~	prompt_domain_share	0.958	neg_max_logprob_token	0.744	+0.213	[+0.081, +0.331]	0.0005	<i>excluded — confounded</i>

The permutation test recomputes the maximum over all seven registry signals *inside every label shuffle*, so the reported *p*-value already accounts for the post-hoc selection of the best signal.

Confound control. Medicine is excluded by an automatic gate, not by inspection. Real disease names are common English words that tokenize to a single in-context BPE token (*asthma* , *stroke* , *migraine*); invented proper nouns cannot. The in-context token-length gap between the real and invented lists is 0.767 for medicine against a 0.5 threshold, versus 0.133, 0.167 and 0.033 for the other three domains. The experiment prints the flag and reports the restricted result separately (*logs27.txt:1186–1208*).

What this supports, and what it does not. In one of three non-confounded domains the result is robust: the confidence interval excludes zero, the permutation *p* survives Bonferroni correction across the three domains ($\alpha = 0.0167$), and five of seven independent signals clear the strongest baseline. Law is suggestive but not robust. Code fails, consistent with its behaviour throughout this work: its purest slots fire on punctuation and fire an order of magnitude less often than other domains'.

We do not claim a general hallucination detector. We claim a measured instance, in one domain, in which a frozen model's sparse-dictionary state at the moment it reads an entity is more informative about ungroundedness than the model's own next-token uncertainty at the same positions.

Second test. A cloze-style correct-versus-incorrect test was included and returned no usable signal: Pythia-410M produced zero exact-match correct answers across all 360 runs under sampling, leaving a single label class. The experiment's degeneracy check detected this and reported an explicit honest negative rather than propagating undefined AUROC values (*logs27.txt:1161, 1173*).

4. Six Measured Pitfalls

We report six evaluation failures encountered while building a sparse-registry steering and hallucination-detection pipeline on a frozen Pythia-410M. Each one individually produced either a false negative (an effect we wrongly reported as absent) or a false positive (a number we wrongly reported as evidence). All six were caught, and all are quantified from our own runs; log line references are given so that each claim is traceable.

We present them in the order a practitioner is likely to meet them: first the failures that corrupt the *measurement apparatus*, then those that corrupt the *conclusion drawn from it*.

4.1 An uncalibrated intervention magnitude produces a false negative

Symptom. Our first steering experiment on the frozen model biased the two purest slots per domain across a strength grid of $[0, 0.5, 1, 2, 4] \times \theta$, where θ was the empirical top-k selection threshold. Across all four domains and all strengths, the targeted-vocabulary rate was flat at approximately zero. Fluency was healthy throughout (repetition 9–32%, entropy 4.2–5.2 nats), so the null could not be attributed to degenerate generation.

The conclusion we drew, and it was wrong. We recorded the result as a negative: *writing into registry slots does not steer a frozen pretrained model in any measurable direction*. The reasoning appeared sound — the intervention was applied, the model kept generating fluently, and nothing moved.

What was actually happening. We had never measured how large the intervention was *relative to the signal it was competing with*. We added a diagnostic, `delta_ratio`, defined as the norm of the injected residual-stream update over the norm of the residual stream itself:

```
delta_ratio = || gain · (decode(topk(pre + bias)) - decode(topk(pre))) || / || h ||
```

The failed experiment had been injecting a perturbation of roughly **0.2% of the residual stream norm**. The intervention was not ineffective; it was *inaudible*. When we widened the intervention to eight slots per domain and swept a gain factor, the same architecture, the same model and the same slots produced a monotone dose–response curve:

<code>delta_ratio</code>	0%	2.8%	6.1%	7.7%	9.0%	11.4%
targeted-vocabulary rate	0.0%	1.3%	16.0%	22.8%	38.2%	41.7%

The effect had been there the whole time, below the threshold of measurement.

Fix. Report `delta_ratio` (or an equivalent relative-magnitude measure) for every intervention row, and calibrate the strength grid so that it spans the region where the quantity is on the order of a few percent. In our setting the usable window was roughly 6–8%; below ~1% nothing is measurable, and above ~10% fluency degrades.

Transferable lesson. A null steering result is uninterpretable unless the intervention magnitude is reported. "We intervened and nothing happened" and "we did not intervene enough to be detectable" are indistinguishable without it, and the second is the more common of the two.

Evidence: exp4 (all-zero sweep) vs exp4b (`delta_ratio` diagnostic and gain sweep).

4.2 Greedy decoding manufactures the collapse you are trying to detect

Symptom. Our collapse detector flagged *every* non-zero intervention strength, in every domain, in both intervention directions, as having broken fluency. There was no strength small enough to escape it.

The conclusion we drew, and it was wrong. We recorded that the intervention mechanism destroys generation coherence at any dose, and therefore that no clean operating window exists.

What was actually happening. Generation used deterministic argmax decoding. Small language models are well known to fall into repetition loops under greedy decoding, and ours did — *before any intervention at all*. At `strength = 0.0`, with the steering bias identically zero and the intervention path a bit-exact no-op, the measured repetition rate was:

```
BOOST, strength = 0.0 : rep = 77.4% (all four domains, shared neutral prompt)
SUPPRESS, strength = 0.0 : rep = 69.8% - 88.9%
```

(logs16.txt:240–258 and 271–289)

Our collapse threshold was 50%. The unintervened baseline was already at 77%. The detector was not measuring the effect of our intervention; it was reporting a property of our decoding strategy.

Fix. Replace greedy decoding with sampling (`temperature = 0.8` , `top_p = 0.9`). Under the corrected decoder the same baseline condition measured 5.0–18.6% repetition, and across all 48 intervention rows — every domain, every strength, both directions — **zero rows were flagged as collapsed**.

Cost. Five runs were contaminated before this was found, and two results we had reported as positive had to be withdrawn, because the old detector's threshold logic had silently exempted the lowest non-zero strengths from the collapse check.

Transferable lesson. Measure your control condition first. If the unintervened baseline already fails your quality gate, the gate is measuring your pipeline, not your intervention.

4.3 Metric mismatch: the intervention worked, and we were looking elsewhere

Symptom. After the magnitude fix, the intervention moved the targeted metric in one domain and left it at exactly zero in the other three.

The conclusion we drew, and it was partly wrong. We recorded that steering works for one domain and fails for the rest, and began looking for properties of the failing domains that would explain it.

What was actually happening. Our metric counted the rate of *distinctive* domain vocabulary — terms selected by relative frequency against the other three corpora (e.g. `subparagraph` , `Secretary` for law). But inspection of the slots we were boosting showed that the purest slots in those domains fire on *function words and punctuation* (`the` , `(` , `shall` , `this` for law; `:` , `)` , `.` , newline for code). Boosting such a slot changes the register and formatting of the text, not its rare content words.

Adding a second metric — the rate of the tokens that *the boosted slots themselves* most strongly fire on — revealed movement of up to tens of percentage points in domains where the original metric had been flat at zero. The intervention had been working in three of four domains; the metric had been measuring the wrong thing in two of them.

Fix. Report both a *concept-defined* metric and a *feature-defined* metric. Their divergence is itself informative: it localizes the mismatch between what the feature encodes and what the evaluator counts.

Transferable lesson. When an intervention shows no effect, the first question is not "why is the mechanism failing" but "is the metric measuring the quantity the mechanism actually moves."

A caution we impose on ourselves. The feature-defined metric turned out to have a serious failure mode of its own, described in §4.6. It should be reported alongside the concept-defined metric as a diagnostic, never used alone as evidence of success.

4.4 Measuring at the wrong token span yields a total null — and the baseline is the clue

Symptom. Our hallucination-detection experiment returned AUROC ≈ 0.5 for every registry signal, at every value of N , on both tests. A complete null.

Why this null was diagnosable. The experiment included a mandatory baseline: the frozen model's own next-token entropy and negative max log-probability, read from its LM head on the same runs. **That baseline was also at 0.5.**

This is the key observation. Had the registry been at chance while the model's own uncertainty signal discriminated, the correct conclusion would have been that the registry carries no information the model does not already expose. But both were at chance simultaneously — and the model's own uncertainty is known to carry *some* signal about unfamiliar entities. Two independent measurement instruments returning chance at the same time indicates that the quantity is not present *where both are looking*.

What was actually happening. Our read-only hook recorded only the final position of each forward pass, which meant we were measuring at the last prompt token and at the generated tokens. But the moment at which a model can be said to encounter an entity it does not know occurs while it *reads that entity*, during the prefill. The entity tokens were never measured at all.

Fix. Record all positions of every forward pass and compute signals over three explicit spans: the entity tokens, the full prompt, and the generated tokens. Under the corrected measurement, on the same model and the same registry, polarity-corrected discrimination at the entity span reached 0.65–0.84 in the non-confounded domains, while the generated-token span remained at chance — exactly as before.

Result. The final automated verdict, comparing against the stronger of four LM-head baselines computed at the same positions, reported a margin of **+0.139** at the entity span (*logs27.txt:1121*).

Transferable lesson. Always include a baseline you expect to work. Its failure is not a nuisance — it is the most informative single bit in a null result. A registry-only null is ambiguous; a *joint* null localizes the problem to the measurement position rather than the instrument.

4.5 A control pinned at the metric floor carries no information

Symptom. Our strongest reported result was a comparison against a purity-matched control: eight slots drawn from *other* domains, greedily matched to the target slots' measured purity (mean absolute purity gap 0.000 for three of four domains) and driven at a matched injected magnitude. The control returned $0.0\% \pm 0.0\%$ at every strength and in every repeat, against $22.8\% \pm 2.7\%$ for the targeted slots. We reported this as decisive.

What an adversarial audit of our own result found. The control's value was **0.0** with standard deviation **0.0** in **18 of 18 runs**. It is therefore *numerically identical* to the raw targeted measurement:

```
specificity_gap = hit(target) - hit(control) = hit(target) - 0 = hit(target)
```

The control added reassurance, not information. Any comparison it licenses is a comparison the raw number already licenses.

A second problem, structural rather than numerical. Inspection of the selected control slot identifiers showed that they were themselves *other domains' target slots*: medicine's controls included code's fourth-ranked target slot, law's second, and literature's seventh. The question the control actually answers is therefore "*if I steer toward a different domain, do this domain's words appear?*" — a test of cross-domain leakage. It is a legitimate question, and the answer is informative. But it does not license the claim we attached to it, because **any dictionary with distinct per-domain directions passes it, including a randomized one.**

Fix. Two changes. First, use a metric with dynamic range in both directions (for example the log-odds of domain-vocabulary mass in the next-token distribution) so that a control can be measurably below, at, or above the target. Second, report multiple controls that rule out different hypotheses; a single control conflates them (see §6.5).

Transferable lesson. A control that cannot move is not a control. Before reporting a control-based claim, check the control's variance: if it is identically zero, the metric is floored and the control is decorative.

Revised claim. We now state this result as: *the effect is specific to the targeted domain's slots as against another domain's slots at matched purity and matched magnitude* — and explicitly not as evidence that the effect requires a learned dictionary.

4.6 A metric that rewards fluency collapse

Symptom. The feature-defined metric introduced in §4.3 produced the single largest specificity margin anywhere in our data: **66.7 percentage points** in the code domain, with a t-statistic of 38.

What was actually happening. That row's generation had `repetition = 95.8%` and `entropy = 0.75 nats` — near-deterministic emission of a single token, flagged as collapsed in all three repeats. The feature-defined vocabulary for code's purest slots consists of punctuation and whitespace (`:`, `)`, `.`, newline). A generation that degenerates into repeating one such token scores near-perfectly on a metric that counts exactly those tokens.

The metric was not measuring successful steering. It was measuring collapse, and rewarding it.

Why this matters beyond one row. The result we had described in our own records as the cleanest in the project — a 52.2% versus 1.7% separation in the law domain — is produced by the same metric, on slots whose feature vocabulary is likewise function words and punctuation. We withdraw it from our primary analysis.

Independent confirmation. This conclusion was reached from a single collapsed row. A later experiment (§6.6.3) tested it on entirely new data, with the metric held fixed to the targeted domain's vocabulary across four arms. In one domain a *random direction* scored 42.3 against the targeted arm's 20.7; in another, a purity-matched control equalled the targeted arm. A metric that ranks a null above the treatment cannot serve as evidence of steering, and the withdrawal stands on independent grounds.

Fix. Gate every metric on a fluency criterion computed on the same generation, and report the collapse rate per condition as a co-primary outcome. Where two conditions have different collapse propensities — which is precisely the case when comparing a trained dictionary against a randomized one — a collapse-rewarding metric cannot be used for comparison at all.

Transferable lesson. Ask of every metric: what score does a degenerate output receive? If the answer is "a high one," the metric is unusable for any comparison in which the conditions differ in their tendency to degenerate.

4.7 Summary

#	Pitfall	Direction of error	Quantified evidence
4.1	Uncalibrated intervention magnitude	False negative	injected 0.2% of residual norm; effect appears from ~6%
4.2	Greedy decoding	False negative	baseline repetition 77.4% at zero intervention
4.3	Metric mismatch	False negative	3 of 4 domains moved on the feature-defined metric only
4.4	Wrong measurement span	False negative	AUROC 0.5 including the baseline; 0.65–0.84 at the entity span
4.5	Control pinned at the floor	False positive	control 0.0 ± 0.0 in 18/18; gap $\equiv$ raw value
4.6	Metric rewards collapse	False positive	66.7 pp margin at entropy 0.75, repetition 95.8%

Four of the six produced false negatives — results we would have published as "the mechanism does not work." Two produced false positives — results we did publish internally as successes and later withdrew. We note that the false negatives were each caught by adding a *measurement*, while the false positives were caught only by an adversarial review of results we already believed (§5).

The single most reusable item in this section is the diagnostic in §4.4: when a null result includes a baseline you expected to work, the joint failure localizes the problem to *where* you are measuring, not to *what* you are measuring with. That observation converted a complete null into the strongest positive result in our pipeline.

5. An Adversarial Audit of Our Own Claims

The pitfalls in §4 were found during development, each by adding a measurement that had been missing. Two of them, however — the floored control (§4.5) and the collapse-rewarding metric (§4.6) — were not found that way. They were found only after the results had been recorded as successes, by deliberately attempting to destroy them.

We describe that process, because it is the part of this work we consider most reusable, and we report its outcome without softening: **four of our own recorded claims did not survive it.**

5.1 Method

Before committing to a further experiment, we submitted the design *and the established results it rested on* to five independent adversarial reviews, each assigned a distinct axis and each instructed to attempt refutation rather than confirmation:

axis	task
prior art	find published work that removes the novelty of the proposed contribution
external source	verify our reading of the paper we were responding to, from its source, and flag any over-reading
implementation	verify every claim we made about our own code, with line numbers, from the code
design	attack the experimental design; quantify anything that would make its result uninterpretable
independent replication of the above	a second executor, given the same brief with the relevant code injected

Two properties of this setup mattered more than the number of reviewers.

First, **the reviews covered our established results, not only the new design.** It is normal to review new code; it is less common to re-open conclusions that have already been recorded as settled. Three of the four claims withdrawn below came from that re-opening.

Second, **reviewers were required to cite evidence they had themselves retrieved** — a line number, a log row, a quoted passage — rather than to summarize our description of it. Two of the four withdrawals were found by recomputing figures directly from our own run logs, which we had summarized correctly but compared incorrectly.

5.2 What did not survive

R1 — "The result replicated across three seeds"

We had recorded a targeted-vocabulary rate of $22.8\% \pm 2.7\%$, described as a replication of an earlier run's 23.5%.

Recomputation from the logs shows the two figures were measured **at different intervention strengths**. At the same strength (2.1868):

run	targeted-vocabulary rate
earlier run	23.5%
later run, three repeats	15.5%, 16.5%, 16.0% → 16.0% ± 0.5%

(logs22.txt:278–280)

A 7.5 percentage-point difference between two bit-identical registries measured at the same dose. Pooling the four available draws gives $\hat{\sigma} \approx 3.8$ pp — the reported ± 0.5 understates the true dispersion by roughly a factor of eight, which is the expected behaviour of a standard deviation estimated from three samples (2 degrees of freedom; $SD(s) \approx 0.52\sigma$).

Further, the three "repeats" varied only the *generation* seed. The training seed was fixed at a single value in every run reported in this work. **No seed replication was performed.**

Withdrawn. The number is reported in §3.3 as a point on a dose–response curve, with the curve's monotonicity — not any single cell — as the evidence.

R2 — "The control returned exactly zero, therefore the effect is specific"

Discussed in §4.5. Two independent problems.

The control's value was 0.0 with standard deviation 0.0 in 18 of 18 runs, so the difference statistic was numerically identical to the raw targeted value and carried no additional information.

And the control slots were themselves *other domains' target slots* — medicine's controls included code's fourth-ranked, law's second-ranked and literature's seventh-ranked target slots. The comparison therefore tests cross-domain leakage. It is a real test with a real answer, but it is passed by any dictionary with distinct per-domain directions, including a randomized one, and so cannot support a claim about learned structure.

Restricted, not withdrawn. We now state the result as specificity *against another domain's slots at matched purity and matched magnitude*, and not as evidence that a learned dictionary is required. §6.5 sets out the additional controls that would be needed for the stronger claim.

R3 — "The cleanest separation in the project"

A 52.2% versus 1.7% separation in the law domain, on the feature-defined metric.

The same metric produces its largest value anywhere in our data — 66.7 percentage points in the code domain — on a generation with repetition 95.8% and entropy 0.75 nats, flagged as collapsed in all three repeats. The metric counts the tokens the boosted slots fire on; for law and code these are function words and punctuation; a degenerate repetition of such a token scores near-perfectly.

Withdrawn from primary analysis. The metric is retained only as a mismatch diagnostic (§4.3), always reported alongside repetition and entropy.

R4 — "Training is not reproducible even at fixed seed"

We had carried this forward as an established property of the pipeline, and had been reading single-run figures as draws from a wide distribution accordingly.

It is false for the regime in which the results of this paper were obtained. Across ten independent sessions and seven distinct codebases, the purity gap was identical to sixteen significant figures and per-slot firing counts were identical as integers.

The non-reproducibility we had observed (final dead-slot fraction ranging 11% to 48% at fixed seed) belonged to an **earlier architecture that no longer exists** in this pipeline: a dictionary trained *jointly* with a from-scratch transformer over 6000 steps. Training only a dictionary on a fully frozen pretrained model is a far better-conditioned problem, and in our runs it was deterministic.

Withdrawn. The correction cuts both ways: we had been applying an inapplicable caution, and simultaneously had never measured the seed variance that does matter. §3.2 states this.

R5 — The decision rule's false-positive rate was uncontrolled

Not a single claim but the rule that generated several. Our automatic verdict declared a "clean window" where the targeted condition exceeded both its baseline and its control by more than twice the pooled standard deviation, with $n = 3$ per group.

With $n = 3$, that threshold is equivalent to $t > 2.45$ on 4 degrees of freedom — a two-sided $p \approx 0.070$ — and it was applied across 4 domains $\times$ 2 metrics $\times$ 6 strengths = **48 tests with no correction**. The expected number of spurious clean windows under the null is **3.4**, and the family-wise error rate, if any single window is treated as a success, is **0.97**.

Our run reported clean windows in three of four domains. That count is fully consistent with noise under this rule.

Consequence. Verdicts produced by that rule are treated as exploratory throughout this paper. The one confirmatory claim we retain (§3.5) uses a pre-specified aggregation, an entity-level independent unit, a permutation test that recomputes the maximum over signals inside each shuffle, and Bonferroni correction across domains.

5.3 What survived, and why the distinction matters

Six findings survived unchanged (§3.1–3.5, and the suppression-saturation measurement). They share a property that the withdrawn claims did not: each rests either on a **monotone trend across several points** or on a **quantity measured identically across many independent runs**, rather than on a single cell compared against a single reference.

The withdrawn claims, by contrast, were all of the form *"this number is large and this other number is small."* That form is exactly what a floored control, an unmatched dose, an uncorrected multiple comparison, or a collapse-rewarding metric will produce for free.

5.4 A note on cost and on what this section is for

The audit cost approximately one working session and no additional model training or GPU runs, though it consumed a substantial number of language-model API calls, as the reviews were model-assisted. It removed four claims, one of which we had described in our own records as the cleanest result in the project.

We report it in full for two reasons. The first is simply that the claims were wrong and the record should say so. The second is that the *distribution* of what it caught is informative: the four false negatives in §4 were each found by adding a measurement, but **both false positives were found only by attacking results we already believed**. No amount of additional instrumentation would have surfaced them, because the instruments were reporting correctly — a control really was returning zero, and a metric really was returning 66.7 points. What was wrong was the inference drawn from those readings.

We suggest that a deliberate adversarial pass over *settled* results, not only over new code, belongs in the standard practice for causal-specificity claims, and that its cost is low relative to the cost of publishing a result that a reviewer will overturn.

6. A Control Protocol for Causal-Specificity Claims

§4.5 and §5.2 established that our own control, despite being carefully constructed, licensed a weaker claim than we attached to it. This section sets out what we think a control for a causal-specificity claim has to do, organized around what each control class rules out.

The framing is deliberately modest: we propose a protocol and identify the two components of it that we did not find in use in this literature. §6.6 reports the result of running four of its five controls, and §6.5 and §6.7 state what remains untested and why.

6.1 The claim being controlled

A causal-specificity claim has the form:

Intervening on feature set F changes behaviour B , and the change is attributable to F rather than to the act of intervening.

The second clause is where the work is. It decomposes into a sequence of progressively stronger negations, and a single control cannot establish more than one of them.

6.2 Control taxonomy

control	rules out	used in
no intervention	"nothing moves anyway"	most steering work
random direction, magnitude-matched	" any vector of this size steers "	established practice [9]
randomized dictionary (frozen / soft-frozen decoder)	" learning the dictionary is unnecessary "	recent sanity-check work [1]
other concepts' features, quality-matched	" any good feature steers "	not found in this literature
shuffled decoder (same columns, permuted assignment)	" any learned column steers "	not found in this literature

The first three negate hypotheses about the **mechanism**: whether anything happens, whether direction matters, whether training matters. The last two negate hypotheses about **attribution**: whether *this* feature, as opposed to any comparable one, is responsible.

Neither group substitutes for the other. A result that clears a random-direction control has shown that direction matters; it has not shown that the *particular* direction is the concept it is labelled with.

6.3 The unclaimed component: quality-matched controls

A search across the dictionary and steering literature returned no instance of a control set selected by **matching on a measured feature-quality score**. We claim only that we did not find one in this literature; permutation- and matching-based controls are long established elsewhere, including in saliency-map sanity checks. Existing controls are either constructed nulls (random directions, permuted signs, subspace projections) or ablated training variants. Controls drawn from *real learned features chosen to be comparable on a measured axis* appear not to have been used.

The construction is simple. For each target feature, select from the pool of features belonging to other concepts the one whose measured quality is closest, greedily and without replacement, and record the per-feature matching gap so that

match quality is auditable from the log alone. In our runs this produced a mean absolute purity gap of 0.000 for three of four domains.

And it is not sufficient on its own. Two failures observed in our own implementation must be designed out:

1. **The metric must have dynamic range in both directions.** Our control returned exactly the metric's floor in every run, which made the difference statistic numerically identical to the raw targeted value (§4.5). A control that cannot move is decorative. A metric such as the log-odds of domain-vocabulary mass in the next-token distribution admits movement in both directions and makes a floored control impossible.
2. **Matching must cover the variables known to be causal.** We matched on purity and used firing frequency only as a tie-breaker, leaving per-slot frequency gaps in the thousands — while our own analysis had identified firing rarity as the mechanism behind one domain's failure to steer. A variable known to drive the outcome cannot be left unmatched. We propose matching jointly on quality and on log firing frequency.

6.4 Proposed protocol

1. **Report intervention magnitude** for every row, relative to the signal intervened upon, and **match it across conditions** to a tight tolerance. Compare dose–response *curves* rather than a single operating point selected after seeing the outcome. Our own verdict rule admitted a 35% magnitude mismatch, which at the measured slope of the dose–response curve is worth up to 21 percentage points of the metric — larger than the effect being measured.
2. **Use a two-sided metric** with range in both directions, and gate it on a fluency criterion computed on the same generation. Report collapse rate per condition as a co-primary outcome (§4.6).
3. **Report at least three controls:** magnitude-matched random direction, quality-matched other-concept features, and shuffled decoder. They rule out different things and the reader needs all three.
4. **Match on every variable known to be causal**, not only the headline one.
5. **Vary the training seed, not only the generation seed**, and report the seed-level variance separately. Our own repeats varied only the generation seed and were reported as replication (§5.2, R1).
6. **Pre-specify one primary endpoint** and correct the rest for multiple comparisons. Our unadjusted rule had a family-wise error rate of 0.97 across 48 tests (§5.2, R5).
7. **Diversify the prompt.** Our steering measurements used a single degenerate context, leaving prompt variance entirely unmeasured.

6.5 What remains untested, and two objections we cannot yet answer

We have since run four of the five controls in §6.2; the results are in §6.6. What remains untested is the randomized-dictionary arm, for the reason given in §6.7, and the two-sided metric of §6.3, which is not implemented. The protocol is therefore partly validated: its control taxonomy has been exercised and produced a result the single-control version could not license, while its dose-matching requirement was measurably violated in our own run (§6.6.2) — which is itself evidence that the requirement needs enforcing rather than assuming.

Two objections are on record and we do not consider them answered:

Magnitude may be the wrong matching variable. Recent work [10] argues that matching the norm of a perturbation fixes its Fisher norm while leaving its alignment with the behavioural gradient free — so norm-matched directions "need not be comparably dosed" in the sense a specificity audit requires. If this is right, `delta_ratio` matching is necessary but not sufficient, and an off-target-KL criterion should be reported alongside it.

Quality may be the wrong matching axis. Work [11] separating "input features" (which reflect input activations) from "output features" (which causally influence output tokens) reports substantial steering gains from selecting on the latter. Purity, as we compute it, is an input-side score. A reviewer is entitled to ask why controls should be matched on an input-side quantity when the claim being controlled is about output-side effect. We think the answer is that matching should be on the axis used to *select* the targets — and we selected on purity — but this is an argument, not a result.

6.6 Running the taxonomy: results

§6.2 set out five control classes and what each rules out, and §6.5 noted that two of them had no data in our pipeline. We ran them. This section reports the result, which strengthens one claim, settles a second, and introduces a measured caveat that applies to all of them.

Design. The identical registry (training is bit-reproducible in this regime, §3.2) with four arms per cell — domain × strength × repeat:

arm	construction
targeted	bias the domain's eight purest slots
control	bias eight purity-matched slots belonging to <i>other</i> domains
random_dir	bypass the dictionary entirely: inject a random unit vector scaled to the norm the targeted arm injected in that same cell , redrawn independently per repeat
shuffled_dec	bias the same target slot ids , but decode through a permuted slot-to-column assignment of the same decoder matrix

288 runs. The encoder and top-k selection are untouched in every arm; only the write path differs.

6.6.1 Result on the concept-defined metric

Medicine, the one domain in which this metric has dynamic range:

strength	targeted	control	random_dir	shuffled_dec
0.00	0.0	0.0	0.0	0.0
1.64	1.3	0.0	0.0	0.0
2.19	16.0	0.0	0.0	0.0
2.73	22.8	0.0	0.0	0.0
3.28	38.2	0.0	0.0	0.0
4.37	41.7	0.0	0.0	0.0

All three controls return zero at every strength, while the targeted arm rises monotonically to 41.7%.

The `shuffled_dec` result is the informative one. That arm biases the *same slot identifiers*, at approximately the same injected magnitude, through the *same set of decoder columns* — differing only in which column each slot writes through. It returns zero. Slot quality and the columns being learned are therefore not sufficient on their own to produce the effect; what distinguishes the targeted arm is **the specific slot-to-column correspondence**. Intervention magnitude is not fully excluded — see the dose mismatch in §6.6.2.

This is the strongest attribution our pipeline supports. It is stronger than the claim we made before running these arms (§5.2, R2), which the purity-matched control alone could not license — and weaker than it would be had the arms been equally dosed.

Literature shows the same ordering at much smaller magnitude (targeted 3.5–3.8 against controls 0.0–0.8). **Law and code** sit at the metric floor in all four arms and are uninformative here, consistent with §4.3.

6.6.2 A measured caveat: the arms are not equally dosed

Our own dose-match check failed on **every cell**, at a 5% tolerance:

```

medicine 2.7335 : gap 19.1%   targeted 7.708% | control 7.479% | random_dir 6.233% | shuffled_dec 6.651%
law      1.6401 : gap 29.0%
...
all cells, gaps 18%-29%

```

The targeted arm consistently injects a larger perturbation than the controls at the same nominal strength. At the dose-response slope measured in §3.3 — roughly 7.9 points of the metric per point of `delta_ratio` — a 19% relative gap at `delta_ratio ≈ 7.7%` corresponds to a bias of order **11 percentage points**.

We report this rather than suppress it. It does not account for a separation of 41.7 against 0.0, but it is large enough that a smaller effect measured this way would be uninterpretable, and it invalidates any fine comparison *among* the three controls.

Consequence for the protocol. Matching the nominal strength across arms is not sufficient; each arm must be calibrated to a target `delta_ratio` individually. Our §6.4 already required a tolerance of 0.05; this run shows that requirement is not automatically satisfied by construction, and must be enforced by per-arm calibration rather than assumed.

6.6.3 The feature-defined metric fails, independently

§4.6 withdrew the feature-defined metric from primary analysis on the grounds that it rewards degenerate output, reasoning from a single domain in an earlier run. This experiment tests that conclusion on entirely new data, and confirms it.

At strength 2.19, with the metric fixed to the *targeted* domain's slot vocabulary in all four arms:

domain	targeted	control	random_dir	shuffled_dec
medicine	30.0	31.2	14.8	1.0
literature	20.7	3.7	42.3	24.8
code	88.7	34.7	11.8	70.7

In medicine the purity-matched control **equals** the targeted arm. In literature the *random direction* **exceeds** it. In code the shuffled decoder comes close to it.

The one apparent anomaly in the concept-defined table resolves the same way: `code / random_dir / 4.37` averaged 9.7% because a single repeat returned 29.0% with repetition 86.4% and entropy 1.156 nats — flagged as collapsed.

A metric on which a random direction outscores the intended intervention in a whole domain cannot be used as evidence of steering. We had reached that conclusion by inspection of one collapsed row; it now rests on an independent run in which the metric ranks a null above the treatment.

6.6.4 Completed taxonomy

control	rules out	our result
no intervention	"nothing moves anyway"	✓ 0.0 at <code>strength = 0</code>
random direction, magnitude-matched	"any vector of this size steers"	✓ ruled out (0.0)
other concepts' features, quality-matched	"any good feature steers"	✓ ruled out (0.0)
shuffled decoder	"any learned column steers"	✓ ruled out (0.0)
randomized dictionary	"learning the dictionary is unnecessary"	X not run (§6.7)

What we can now claim. On the concept-defined metric, in one domain, and subject to the dose mismatch of §6.6.2, the pattern is most consistent with the steering effect arising from the specific slot-to-decoder-column correspondence rather than from slot quality or from the columns merely being learned. Because the controls were systematically *under-*dosed relative to the targeted arm, we cannot fully separate magnitude from correspondence on this data.

What we still cannot claim. That a *trained* dictionary is required. That question needs the randomized-dictionary arm, which we did not run for the reason given in §6.7.

Standing limitations on the above. Dose mismatch of 18–29% across arms; one domain with dynamic range on the metric; three repeats per cell; a single training seed.

6.7 Why we did not run the randomized-dictionary arm

The natural fifth arm — a dictionary trained with a randomly initialized and constrained decoder — was designed and then dropped before execution, for a reason we state because it may save others the run.

Under a cosine constraint of $\tau = 0.8$ to a random initialization, a decoder column can carry at most 60% of its magnitude along the direction an unconstrained column would learn. At the dose–response slope of §3.3, that amplitude penalty alone predicts a gap of roughly 13 percentage points between arms — before any question of dictionary quality enters. Our design would then have been unable to distinguish "learning matters" from "the constrained arm was quieter," and at a single training seed per arm its minimum detectable effect exceeded the effect it predicted.

Separately, our expansion factor of 4 makes random dictionaries measurably less expressive than at the expansion factors used in the work reporting that randomized baselines match trained ones. A negative result for the randomized arm at expansion 4 would not transfer upward, and we would not have been entitled to present it as a response to that work.

The arm remains the right experiment; it needs an amplitude-matched null arm, multiple training seeds, and a higher expansion factor to be interpretable.

7. Limitations

We state these at the level of detail we would want from a paper we were reviewing. Several of them are load-bearing enough that a reader should treat the corresponding claims as provisional.

7.1 Scope

One model, one size. Pythia-410M throughout, with a single earlier comparison point at 160M. The scale trend we report in §3.2 rests on two points and is reported as a direction, not a trend.

One dictionary architecture. TopK only. We did not test JumpReLU, BatchTopK, gated variants, transcoders or crosscoders.

Expansion factor 4. 4096 slots over 1024 dimensions. Much recent dictionary work operates at expansion 32 or higher, where random dictionaries are measurably more expressive. Conclusions drawn at expansion 4 do not transfer upward, and in particular any comparison against randomized-dictionary baselines would be biased in our favour at this expansion factor. We do not make such a comparison for that reason.

Four domains, one language. Domains are coarse (medicine, law, code, literature) and English-only. Slot "concepts" are only as fine-grained as the partition used to measure them.

7.2 The most important limitation

We measured domain vocabulary, not correctness.

Every steering result in this paper measures the rate at which domain-associated tokens appear in the continuation. At no point did we measure whether the model's answers became more *accurate*. We have no evidence that intervening on a domain's slots improves task performance, and it would be a misreading of this work to conclude that it does.

The intervention demonstrably changes register, vocabulary and formatting. Whether it changes what the model knows or gets right is untested here.

7.3 Statistical limitations

Seed variance is unmeasured. All runs used a single fixed training seed. The training is bit-reproducible in this regime (§3.2), which means our repeats carried no information about seed-level variability. No claim rests on stability across seeds, and none should be read as doing so.

Small samples in the detection experiment. Thirty real and thirty invented entities per domain. Confidence intervals are correspondingly wide, and the result that survives correction does so in one domain of three.

The exploratory verdict rule. The automatic rule used through most of this work corresponds to an uncorrected $p \approx 0.070$ applied 48 times (§5.2, R5). Every verdict it produced is treated as exploratory. Only the analysis in §3.5 is confirmatory, and it is pre-specified, entity-level, permutation-tested with the maximum recomputed inside each shuffle, and Bonferroni-corrected.

One domain excluded by confound gate. Medicine, the numerically strongest domain in the detection experiment, is excluded automatically because real disease names are common single-token English words and invented proper nouns cannot be. This is the correct action but it removes the strongest data.

7.4 Measurement and implementation caveats

No attention mask on the padded prefix. Generation uses left padding with an end-of-text token and no attention mask, so the model attends to pad positions. This is applied identically across all conditions and both to registry signals and to baselines, so it is not a differential bias — but it is a departure from best practice and we state it rather than leave it implicit.

Decoder renormalization is periodic, not continuous. Decoder columns are renormalized at intervals during training rather than every step, and the saved model is one optimizer step past the last renormalization. No decoder-norm diagnostic was logged. For the results reported here this affects nothing we measure, but it would need fixing before any cross-condition comparison that depends on injected magnitude being comparable by construction.

Control arms were not equally dosed. In the four-arm experiment of §6.6 the injected magnitude differed between arms by 18–29% at the same nominal strength, against a 5% tolerance, in every cell. At the measured dose–response slope this is worth of order 11 percentage points of the metric. It does not account for the separation we report (41.7 against 0.0), but it invalidates fine comparisons among the controls and would make a smaller effect uninterpretable. Per-arm calibration to a target `delta_ratio`, rather than a shared nominal strength, is required.

The feature-defined metric is retained only as a diagnostic. §4.6 shows it is maximized by degenerate output. It appears in this paper as evidence of metric mismatch and nowhere as evidence of successful steering.

7.5 Relation to prior work

This work is an independent replication of established findings, with tighter controls in some places and, as §5 documents, looser ones in others than we had believed. It is not a novelty claim.

Two prior results in particular anticipate parts of it: sparse-dictionary directions that detect whether a model recognizes an entity, reported with causal interventions we did not perform; and a decomposition of causal inertness into read- and write-directional components, which is the mechanism our own reasoning in §6.2 appeals to. Both predate this work and we cite them as such (§8).

The one component we could not find precedent for is the quality-matched control of §6.3, together with the shuffled-decoder control of §6.2. Both were run (§6.6); the quality-matched control, taken alone, returns the metric floor and carries no information, while the shuffled-decoder control is the one that licenses the attribution claim we make.

7.6 A note on the research log

The repository includes the project's working research log, in Arabic (`research-log.ar.md`), with an English translation alongside it (`research-log.en.md`); the Arabic file is the original and governs where the two differ, and the translation is provided for accessibility only. The log is an unedited development diary, written informally and in real time, not a polished document; it is included as evidence of the process described in §5, including the four withdrawn claims. Executor and reviewer roles in it are referred to by neutral labels rather than by product name, because the incidents recorded there are single-sample, time-bound, and in several cases attributable to transport or tooling rather than to any model's behaviour.

7.7 What would change our conclusions

- A run at expansion 32 could invalidate any comparison we make against randomized-dictionary results.
- Measuring seed-level variance could widen every interval reported here.
- Implementing the two-sided metric of §6.3 could show that our surviving specificity result is smaller than reported, since the floored metric cannot express a control performing *worse* than chance.

- A test of task accuracy under steering could show that the intervention we characterize as control is cosmetic.

8. Related Work

We group prior work by its relation to this paper: work that motivates it, work that anticipates its findings, and work that objects to its proposed method. We place the anticipating work first and prominently, because two of those results substantially overlap with ours and it would be misleading to bury them.

8.1 Work that anticipates our findings

Entity-recognition directions and hallucination [4]. Sparse-dictionary directions at entity tokens have been shown to encode whether a model can recall facts about that entity — a form of self-knowledge about its own capabilities — and to be causally relevant, in that intervening on them can induce refusal on known entities or hallucinated attributes on unknown ones.

Our §3.5 recovers the detection half of this independently, on a different model, with a baseline comparison at the same token positions that we do not believe has been reported in that form. We do not recover the causal half: we performed no intervention in the detection experiment. **This prior result is stronger than ours and predates it**, and we cite it as the primary reference for the finding rather than presenting §3.5 as new.

Read- versus write-directional causal inertness [8]. A decomposition of feature causal inertness by direction — separating whether a feature can be *read from* versus *written through* — has been reported, together with the observation that a large fraction of features passing a correlational recovery criterion are causally inert. This is the same mechanism our §6.2 reasoning appeals to when distinguishing controls that test the encoder path from those that test the decoder path. We did not originate that distinction.

8.2 Work that motivates this paper

Randomized dictionary baselines [1]. Recent sanity-check work constructs dictionary variants with randomized and frozen encoders or decoders and reports that the soft-frozen variant matches fully-trained dictionaries on interpretability, sparse probing and causal editing at matched sparsity, concluding that dictionaries in their present form do not reliably decompose model internals.

We note two properties of that comparison relevant to reading it, both visible in the reported tables. First, the three headline pairs correspond to *different* baseline variants — the causal-editing figure to the soft-frozen decoder, the sparse-probing figure to a frozen decoder — so each is a maximum over five variants presented as a single result. Second, the causal-editing metric — the RAVEL disentanglement score [6] — has a floor: a variant with zero explained variance and chance-level probing still scores approximately 0.485. Read against that floor, the headline comparison of 0.73 against 0.72 becomes 0.24 against 0.23, and in two of three model-layer settings the fully-trained dictionary is itself close to the floor. This does not contradict the paper's conclusion — the trained model's above-floor margin is indeed largely matched — but it substantially changes the effect size a reader takes away, and it is not stated in the paper's limitations.

Benchmark reliability [2, 7]. An audit of a widely used dictionary benchmark suite [7] finds that two of its metrics fail multiple evaluation lenses at their canonical settings and should not be used, recommends a third, and explicitly does not propose replacements. Separately, automated interpretability metrics have been shown not to distinguish trained from randomly initialized transformers [3].

Together these establish the gap this paper addresses: there is no accepted causal discriminator for dictionary quality, and the diagnosis is better developed than the remedy.

8.3 Work on controls and intervention magnitude

Magnitude-matched controls [9]. Matching a control perturbation to the same residual-stream distortion as the intervention is established practice, and has been used to show that steering collapse is direction-pattern-dependent rather than magnitude-dependent. Our `delta_ratio` reporting (§4.1) is an instance of this practice, not a contribution; we report it because our failure to apply it produced a false negative, not because the idea is new.

Structural matching of controls [13]. Matched-random-control protocols that hold structural properties constant while varying only concept alignment have been used in attribution-graph work, with the explicit warning that without such matching the labelled-versus-random gap is easy to misstate. This is the closest precedent we found for the *philosophy* of §6.3, though applied to a different object.

Specificity as side-effect measurement [12]. Work on inference-time intervention defines specificity as the preservation of unrelated behaviour — general capability, related-behaviour retention, robustness under distribution shift. This is a different quantity from the attribution specificity of §6.1: it asks whether the intervention broke anything else, not whether the effect is attributable to the intervened feature. That work does not employ matched controls at equivalent magnitude, which is the gap §6.3 addresses.

Feature selection for steering [11]. Separating features that reflect input activations from those that causally influence output tokens, and selecting on the latter, has been reported to improve steering substantially. This is the basis of the objection we record in §6.5: purity is an input-side score, and a reviewer may reasonably ask why controls should be matched on it.

8.4 Work on steering and task performance

Inference-time intervention along truth-correlated directions has been shown to improve accuracy on a truthfulness benchmark substantially [5], establishing that activation-level intervention can change task performance and not only style.

We flag this specifically because it marks the boundary of our own claims: we did not measure accuracy under intervention (§7.2), and the existence of that prior result means the question is neither open nor answered by us.

8.5 Positioning

Relative to this literature, the present work is an independent replication with a documented failure catalogue. Its contributions are the six measured pitfalls (§4), the adversarial audit of its own results (§5), the control taxonomy (§6.2), and two control constructions we did not find in use in this literature (§6.3) — of which the shuffled-decoder control produced the attribution result in §6.6 that the quality-matched control alone could not license.

9. Conclusion

We set out to build a numbered concept registry inside a frozen language model and to test whether its internal state could be used for control and for diagnosis. The mechanism works, within limits we can now state precisely. But the more durable output of the work turned out to be the record of how often, and how quietly, it appeared to work when it did not.

What we can state. Slots in a sparse dictionary trained on a frozen model's activations specialize by domain without supervision, and the separation improves with model scale. Writing into a domain's slots changes generation monotonically in the injected magnitude, with a usable window of roughly 6–8% of the residual stream norm. In one domain, on a concept-defined metric, that effect survives three independent controls — a magnitude-matched random direction, quality-matched features from other domains, and a shuffled decoder that preserves the learned columns but permutes their assignment — each of which returns exactly zero. Subject to the dose mismatch of 18–29% we report in §6.6.2, that pattern points to the specific slot-to-column correspondence as the locus of the effect. And a frozen model's dictionary state, read while it processes an entity name, discriminates real from invented entities better than the model's own next-token uncertainty at the same positions, in one of three non-confounded domains.

What we cannot state. That any of this improves accuracy: we measured domain vocabulary throughout and never measured correctness. That suppression is available: it saturates structurally, by a factor of 34. That the results extend beyond a single 410M model, one dictionary architecture, an expansion factor of 4, and a single training seed. That a *trained* dictionary is required: the arm that would answer it was designed, costed, and dropped because its outcome was predictable from geometry alone.

What we would emphasize to a practitioner. Four of our six pitfalls produced false negatives, and each was found by adding a measurement that had been absent — the injected magnitude, the baseline's own behaviour, a second metric, the positions being read. The remaining two produced false positives, and neither could have been found that way. The control really was returning zero; the metric really was returning 66.7 points. What was wrong was the inference, and only a deliberate attempt to destroy a result we already believed surfaced it.

That asymmetry is the thing we would carry to the next project. Instrumentation protects against the failure to see an effect. Nothing but adversarial review protects against seeing one that is not there — and in our case that review cost one working session and no additional model training or GPU runs — though it did consume a substantial number of language-model API calls, since the reviews were themselves model-assisted — against results that had already been recorded as settled.

A closing note on the shape of the claim. The strongest sentence in this paper is narrow: one domain, one metric, one model, three controls at zero. We arrived at it by removing four broader sentences that we had preferred. We think that trade is the correct one, and that a literature which reported it more often would be easier to build on.

References

All entries below were verified against the arXiv abstract page for the given identifier: author list, exact title, and submission date.

-
- [1] Korznikov, A., Galichin, A., Dontsov, A., Rogov, O., Oseledets, I., and Tutubalina, E. *Sanity Checks for Sparse Autoencoders: Do SAEs Beat Random Baselines?* arXiv:2602.14111, 15 February 2026.
- [2] Chanin, D. *Are Sparse Autoencoder Benchmarks Reliable?* arXiv:2605.18229, 18 May 2026.
- [3] Heap, T., Lawson, T., Farnik, L., and Aitchison, L. *Automated Interpretability Metrics Do Not Distinguish Trained and Random Transformers.* arXiv:2501.17727, 29 January 2025.
- [4] Ferrando, J., Obeso, O., Rajamanoharan, S., and Nanda, N. *Do I Know This Entity? Knowledge Awareness and Hallucinations in Language Models.* arXiv:2411.14257, 21 November 2024. ICLR 2025.
- [5] Li, K., Patel, O., Viégas, F., Pfister, H., and Wattenberg, M. *Inference-Time Intervention: Eliciting Truthful Answers from a Language Model.* arXiv:2306.03341, 6 June 2023. NeurIPS 2023 (spotlight).
- [6] Huang, J., Wu, Z., Potts, C., Geva, M., and Geiger, A. *RAVEL: Evaluating Interpretability Methods on Disentangling Language Model Representations.* arXiv:2402.17700, 27 February 2024. ACL 2024.
- [7] Karvonen, A., Rager, C., Lin, J., Tigges, C., Bloom, J., Chanin, D., Lau, Y.-T., Farrell, E., McDougall, C., Ayonrinde, K., Till, D., Wearden, M., Conmy, A., Marks, S., and Nanda, N. *SAEBench: A Comprehensive Benchmark for Sparse Autoencoders in Language Model Interpretability.* arXiv:2503.09532, 12 March 2025. ICML 2025.
- [8] Bal, M. A. *From Geometric Recovery to Causal Validation: A Reproducible Audit of Sparse Autoencoder Features, from Superposition Geometry to Causal Inertness.* arXiv:2607.12166, 13 July 2026.
- [9] Riegler, M. A., and Torppmann-Hagen, B. S. F. *Steering Grids for Sparse-Autoencoder Features: When a Top-Context Label Names an Activation Regime Rather Than a Causal Axis.* arXiv:2605.03160, 4 May 2026.
- [10] Luo, D., Liang, C., and Xuan, J. *SteerCheck: Attribution Specificity and Alignment Leakage in Activation-Steering Audits.* arXiv:2608.24335, 25 August 2026.
- [11] Arad, D., Mueller, A., and Belinkov, Y. *SAEs Are Good for Steering — If You Select the Right Features.* arXiv:2505.20063, 26 May 2025. EMNLP 2025.
- [12] Goyal, N., and Daumé III, H. *Steering Safely or Off a Cliff? Rethinking Specificity and Robustness in Inference-Time Interventions.* arXiv:2602.06256, 5 February 2026. EACL 2026.
- [13] Birardi, G., and Paulo, G. *Automated Attribution Graph Interpretation via Probe Prompting.* arXiv:2511.07002, 10 November 2025.
- [14] Biderman, S., Schoelkopf, H., Anthony, Q., Bradley, H., O'Brien, K., Hallahan, E., Khan, M. A., Purohit, S., Prashanth, U. S., Raff, E., Skowron, A., Sutawika, L., and van der Wal, O. *Pythia: A Suite for Analyzing Large Language Models Across Training and Scaling.* arXiv:2304.01373, 3 April 2023. ICML 2023.

Datasets

- **PubMedQA** — `qiaojin/PubMedQA`, configuration `pqa_artificial`.
 - **BillSum** — `FiscalNote/billsum`.
 - **CodeParrot** — `codeparrot/codeparrot-clean-valid`.
 - **Project Gutenberg** — `manu/project_gutenberg`, split `en`.
-

Note on [9]

This entry was retitled between arXiv versions. An earlier version circulated as "*Pairwise Matrices for Sparse Autoencoders: Single-Feature Inspection Mislabels Causal Axes*." The title above is the current one; readers encountering the earlier title should note it is the same identifier.